# Processingをはじめよう

第7章

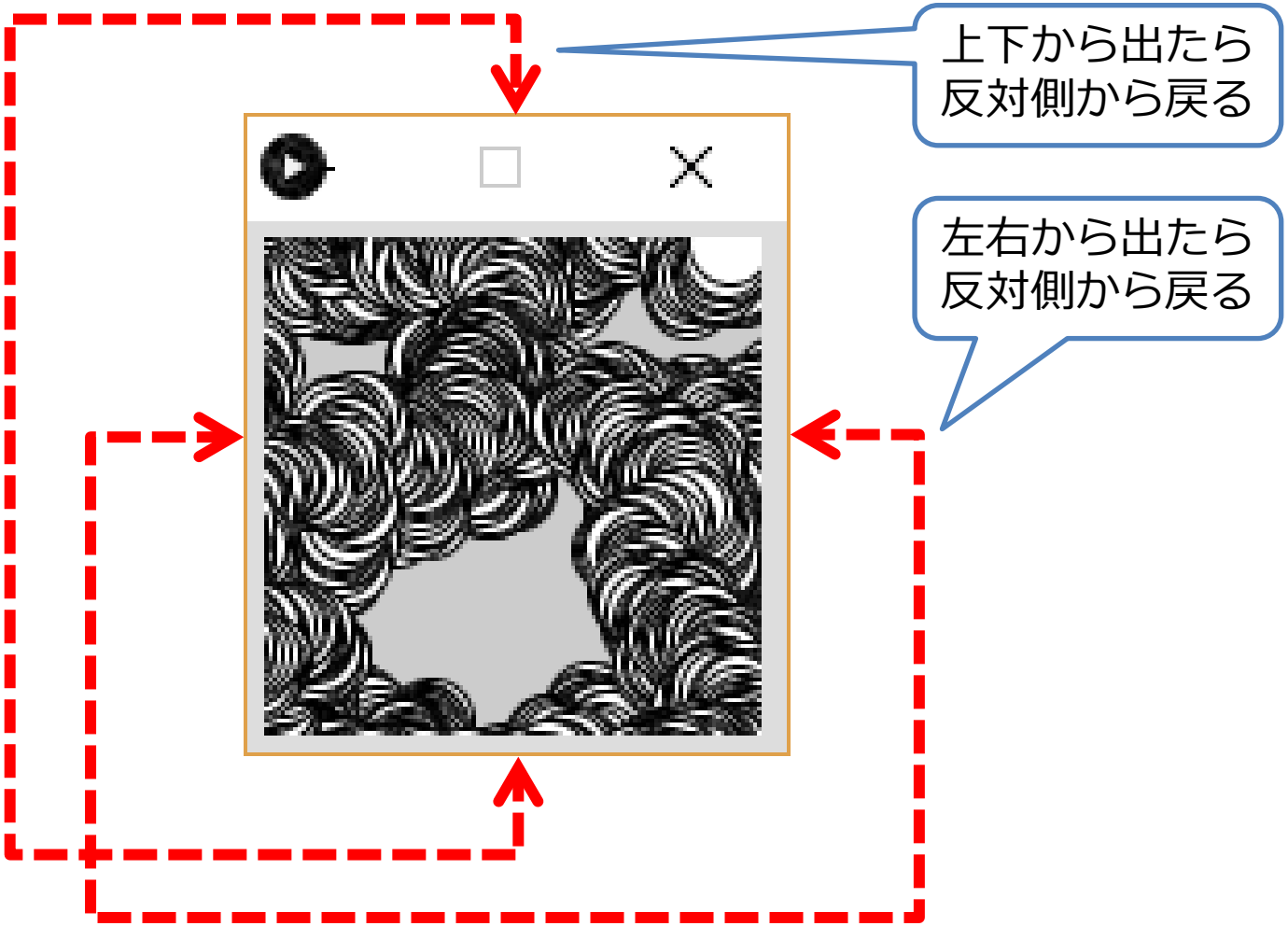
動き　その２

課題　解答例

# Kadai_07_05（必ず提出）

- Example 7-9 と 7-4 を参考に、ランダムに動く円が上下左右から外に出たら、反対側から戻ってくるプログラムを作成する
  - 左から出たら右から戻る、右から出たら左から戻る
  - 上から出たら下から戻る、下から出たら上から戻る
  - 注意：Example7-4は右から出て左から戻ることだけ、左から出て右から戻る・上から出て下から戻る・下から出て上から戻る動きは、自分で考えて追加する

# 実行結果



# プログラム例

```
float speed = 2.5;
int diameter = 20;
float x;
float y;

void setup() {
  size(120, 120);
  x = width/2;
  y = height/2;
}

void draw() {
  x += random(-speed, speed);
  y += random(-speed, speed);

  if(x > width+diameter/2){
    x = -diameter/2;
  } else if(x < -diameter/2){
    x = width+diameter/2;
  }
```

```
  if(y > height+diameter/2){
    y = -diameter/2;
  } else if(y < -diameter/2){
    y = height+diameter/2;
  }

  ellipse(x, y, diameter, diameter);
}
```

# Kadai_07_06（必ず提出）

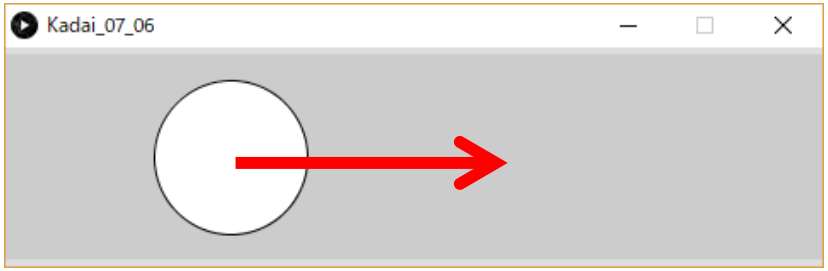
- Example 7-11 を以下のように改良
  - イベント3の設定時間を6000ミリ秒とする
  - 経過時間が6000ミリ秒を超えたら、
    x座標の値を(-1.0,1.0)の範囲のランダムな値を加算する
    (Example 7-9 を参考にするとよい)
  - 3つの条件を指定する順序をよく考える

# 実行結果

currentTime
0 ~ 1999

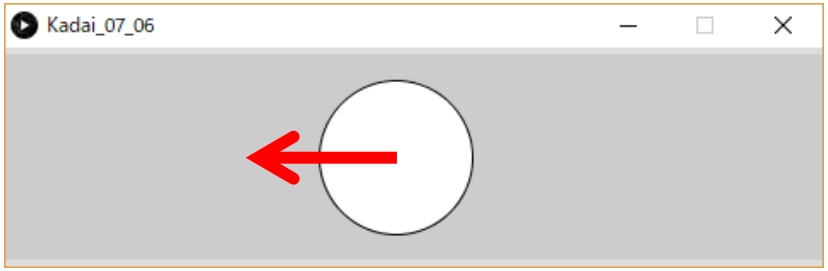
左端で停止

currentTime
2000 ~ 3999



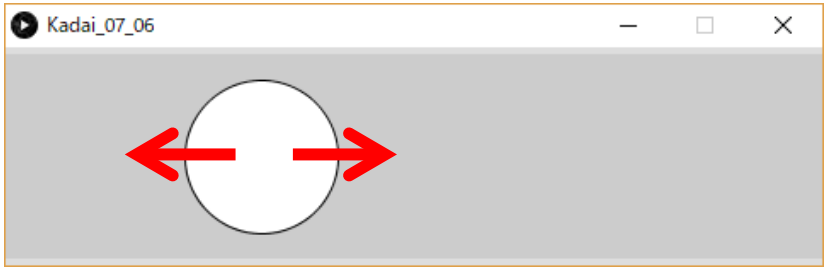

右方向に移動
x += 2.0

currentTime
4000 ~ 5999




左方向に移動
x -= 0.5

currentTime
6000 ~




(-1.0, 1.0)の範囲で
ランダムに移動

# プログラム例

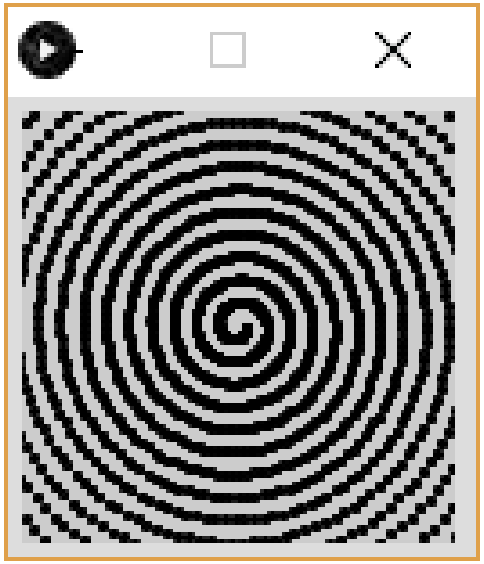
```
int time1 = 2000;
int time2 = 4000;
int time3 = 6000;
float x = 0;

void setup() {
  size(480, 120);
}

void draw() {
  int currentTime = millis();
  background(204);

  if (currentTime > time3){
    x += random(-1.0, 1.0);
  } else if (currentTime > time2) {
    x -= 0.5;
  } else if (currentTime > time1) {
    x += 2;
  }
  ellipse(x, 60, 90, 90);
}
```

# Kadai_07_07（任意）

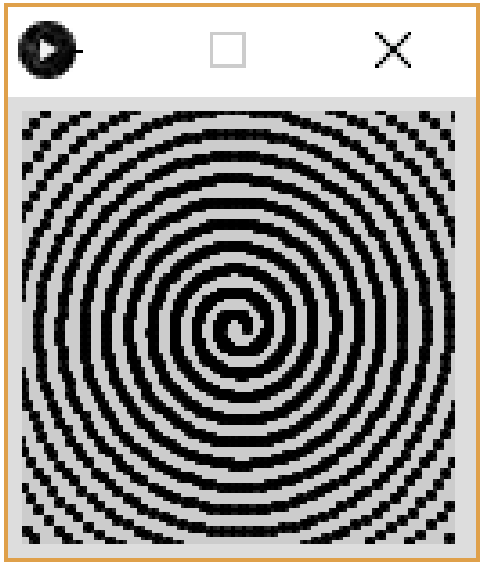
- Example 7-15 を改良して反対回りのらせんを書くプログラムに修正する
  - Example7-15では、時計回りに回転するためangleを0.05加算する処理を行っている
  - 反時計回りに回転するためには？

# 実行結果

Example 7-15
実行結果



Kadai_07_07
実行結果



# プログラム例

```
float angle = 0.0;
float offset = 60;
float scalar = 2;
float speed = 0.05;

void setup() {
  size(120, 120);
  fill(0);
}

void draw() {
  float x = offset + cos(angle) * scalar;
  float y = offset + sin(angle) * scalar;
  ellipse( x, y, 2, 2);
  angle -= speed;
  scalar += speed;
}
```

angleを減らす
(反時計方向に回す)